

Graph problems in **P**
and Search-to-Decision Reductions
CSCI3360 Introduction to Computational Complexity
Tutorial (Week 3)

Chak Fai Yung

Department of Computer Science and Engineering
The Chinese University of Hong Kong

Today's tutorial

- 1 **Section 2.2.2: Directed reachability.**

Decide whether a directed path exists.

- 2 **Section 2.2.3: Bipartite matching.**

Use reachability to improve a matching.

- 3 **Search-to-decision reductions.**

An illustrative example arisen from Competitive Programming.

As in Week 2: describe the algorithm, justify its answer, and bound its time.

Reminders

- ① The solutions to Problem Set 1 are now available.
- ② There will be no tutorial next week (Problemset 3 will still be released).
- ③ Problem Sets 2 are now available.

A common misconception about P

Measure running time in the bit length of the input.

Example: integer factorization. Trial division tests candidate divisors up to $\sqrt{N}$, using $O(\sqrt{N})$ divisibility tests.

This bound is polynomial in the value N . But the length of the binary input is

$$n = \lfloor \log_2 N \rfloor + 1, \quad \sqrt{N} = \Theta\left(2^{n/2}\right).$$

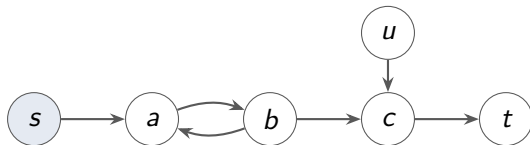
Thus, trial division can require exponentially many tests in the input length. It is not a polynomial-time algorithm.

For **explicitly encoded graphs**, polynomial bounds in $|V|$ and $|E|$ correspond to polynomial bounds in the input length. Why does the encoding make this possible?

Directed reachability

For a directed graph $G = (V, E)$ and vertices $s, t \in V$, decide whether there is a directed path from s to t .

$$\text{PATH} = \{\langle G, s, t \rangle : t \text{ is reachable from } s\}.$$



- s reaches t along s, a, b, c, t .
- s does not reach u . The edge (u, c) points the wrong way.
- A length-zero path makes $\langle G, s, s \rangle$ a yes-instance.

Reachability by marking

On input $\langle G, s, t \rangle$:

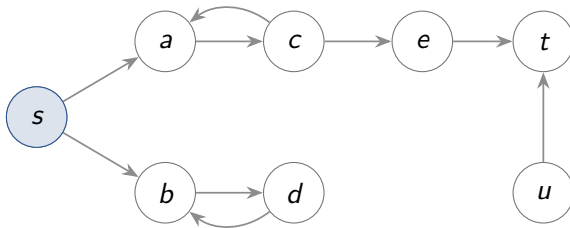
- 1 Set $R \leftarrow \{s\}$.
- 2 Set $R' \leftarrow R$.
- 3 For every edge $(u, w) \in E$, if $u \in R$, add w to R' .
- 4 If $R' = R$, accept exactly when $t \in R$, and halt.
- 5 Otherwise set $R \leftarrow R'$ and return to Step 2.

One scan adds one layer

Test membership in the **old** set R . A vertex added to R' can start an exploration only in the next scan.

A example: initialization

Goal: Determine which vertices are reachable from s .



$$R_0 = \{s\}.$$

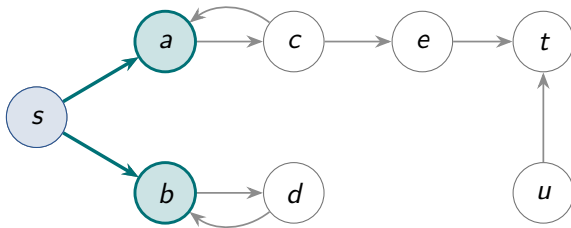
There are two branches, two directed cycles, and an incoming edge (u, t) .

Each scan examines **every edge**, testing its source against the **old set** R . The highlights will show the important updates.

White: unmarked.

Scan 1: discovering both branches

Before the scan: $R = R_0 = \{s\}$, and initially $R' \leftarrow R$.



The edges (s, a) and (s, b) add a and b to R' .

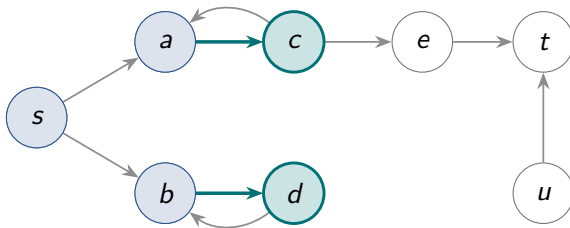
$$R_1 = \{s, a, b\}.$$

The sources a and b are absent from the old set R_0 . Their outgoing edges cannot add c or d during this scan.

White: unmarked.

Scan 2: discovering the next layer

Before the scan: $R = R_1 = \{s, a, b\}$, and initially $R' \leftarrow R$.



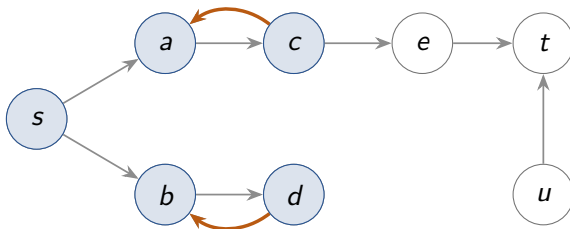
The edges (a, c) and (b, d) add c and d . The edges leaving s lead to vertices already marked.

$$R_2 = \{s, a, b, c, d\}.$$

Although c is now in R' , it is absent from R_1 . The edge (c, e) must wait until the next scan.

Scan 3: edges back to marked vertices

Before the scan: $R = R_2 = \{s, a, b, c, d\}$, and initially $R' \leftarrow R$.

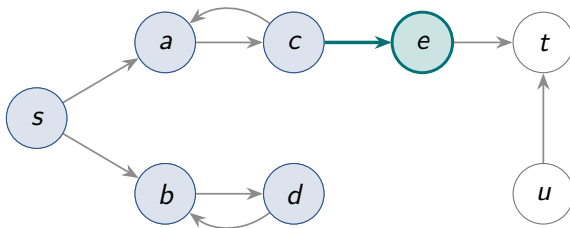


- For (c, a) , the source c is marked, but a is already in R' .
- For (d, b) , the source d is marked, but b is already in R' .

These edges leave R' unchanged. Keep all marks and continue to the next edge in the scan.

Scan 3 continued: the upper branch advances

Continue the **same scan**, still using the old set R_2 .



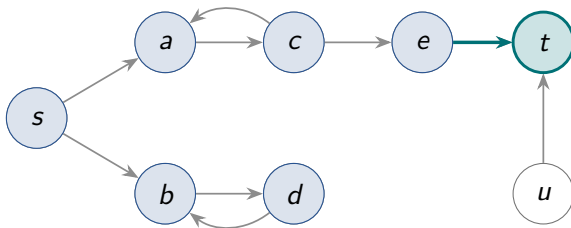
Since $c \in R_2$, the edge (c, e) adds e to R' . Since $e \notin R_2$, the edge (e, t) cannot yet add t .

$$R_3 = \{s, a, b, c, d, e\}.$$

The lower branch contributes no new vertices, while the upper branch still expands the reachable set.

Scan 4: reaching the target

Before the scan: $R = R_3 = \{s, a, b, c, d, e\}$.



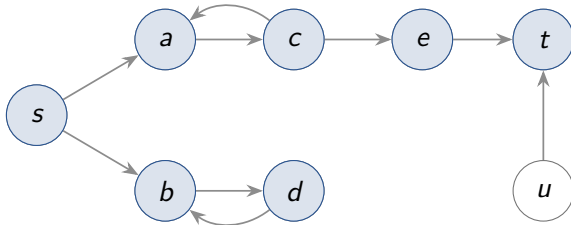
The edge (e, t) adds t .

$$R_4 = \{s, a, b, c, d, e, t\}.$$

A witnessing path is s, a, c, e, t . The vertex u remains unmarked: (u, t) permits movement **from** u **to** t , not from t to u .

Scan 5: stabilization and the answer

Before the scan: $R = R_4 = \{s, a, b, c, d, e, t\}$.



Every edge with a marked source already has a marked destination. The edge (u, t) has an unmarked source and adds nothing, so the algorithm halts as $R' = R_4$

Target t : accept. **Target u :** reject.

The marked sets only grow. Each productive scan adds a new layer; the first scan that adds nothing ends the algorithm.

Why marking is correct

Let R_k be the set of marked vertices after k complete scans. For every vertex v ,

$$v \in R_k \iff v \text{ can be reached from } s \text{ using at most } k \text{ edges.}$$

- **Initially:** $R_0 = \{s\}$, since only s is reachable using zero edges.
- **Each scan adds one layer:**
 - Following an edge from a vertex in R_k reaches a vertex within $k + 1$ steps.
 - Every vertex reachable within $k + 1$ steps is either already in R_k or has an incoming edge from a vertex in R_k .

Once a scan adds no new vertices, all later scans would leave the set unchanged. Therefore, the final set R contains exactly the vertices reachable from s .

$$t \in R \iff \text{there is a directed path from } s \text{ to } t.$$

Halting, running time, and BFS

Write $v = |V|$ and $m = |E|$.

- Each productive scan adds at least one of the $v - 1$ initially unmarked vertices. One final scan detects stabilization.
- Copying the marks and scanning adjacency lists costs $O(v + m)$ graph operations per scan.

$$T_{\text{marking}} = O(v(v + m)).$$

BFS maintains a queue and explores each vertex and edge only once, using $O(v + m)$ graph operations.

Both give polynomial-time algorithms under explicit graph encodings. Polynomial simulation overhead on a Turing machine preserves $\text{PATH} \in \text{P}$.

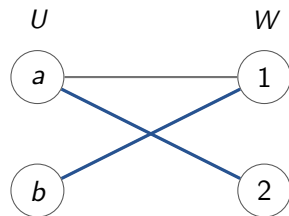
Bipartite matching with a size threshold

A bipartite graph $G = (U, W, E)$ has edges only between U and W , no edge inside U and V .

A **matching** $M \subseteq E$ uses each vertex at most once. Its size is $|M|$.

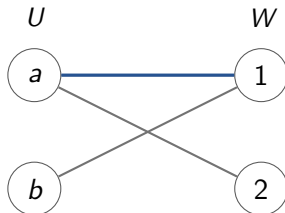
Decision problem: Given G and a binary-encoded integer $k \geq 0$, does G have a matching of size at least k ?

The highlighted matching has size 2. Here $k = 2$ gives **yes**, and $k = 3$ gives **no**.



Maximal and maximum matchings

Maximal, size 1

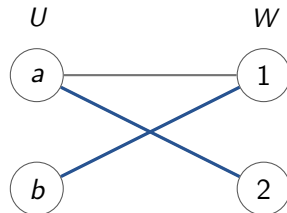


$$M = \{(a, 1)\}$$

No unused edge can simply be added.

Increasing the size may require **replacing** a previously selected edge.

Maximum, size 2

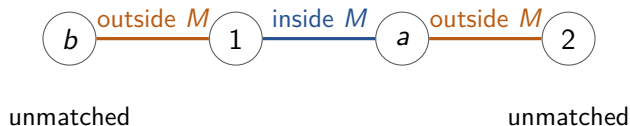


$$M^* = \{(a, 2), (b, 1)\}$$

No matching has larger size.

An augmenting path

Relative to M , an **augmenting path** has unmatched endpoints and alternates between edges outside and inside M .



An augmenting path: the flip

1. Before the flip

$M = \{(a, 1)\}$



b and 2 are unmatched.

2. Remove

$(a, 1)$



1 and a are now free too.

3. Insert

$(b, 1)$ and $(a, 2)$



Each vertex has one selected edge.

— selected - - - unselected

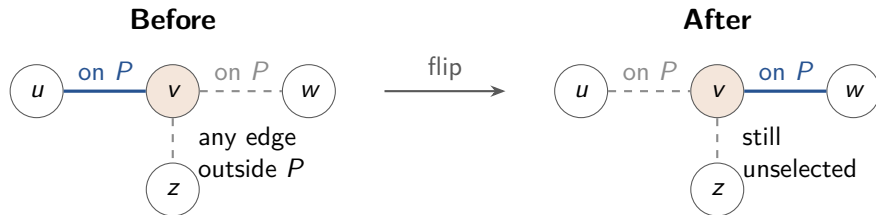
$$\underbrace{\{(a, 1)\}}_M \xrightarrow{\text{remove}} \emptyset \xrightarrow{\text{insert}} \underbrace{\{(b, 1), (a, 2)\}}_{M'}$$

Flip: Toggle membership in the matching along the path.

Why an internal vertex still has one matching edge

Matching condition: At most one selected edge at each vertex.

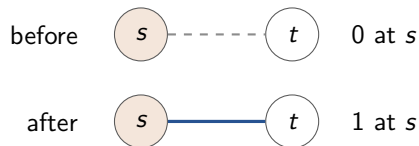
Consider an internal vertex v of the augmenting path P .



- 1 **Before:** Alternation gives v exactly one selected path edge.
- 2 **No hidden conflict:** Every other edge at v is unselected, since M is a matching.
- 3 **After:** Remove the selected path edge and insert the other one. The number of selected edges at v remains **one**.

Endpoints and vertices outside the path

An endpoint s of P

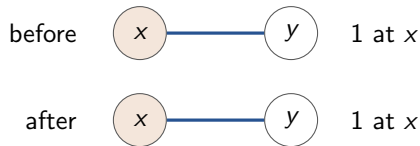


It starts **unmatched**: no selected edge touches s .

The flip inserts its only path edge.
It now has **exactly one** selected edge.

Every vertex still has **at most one** selected edge. Hence M' is a matching.

A vertex x outside P



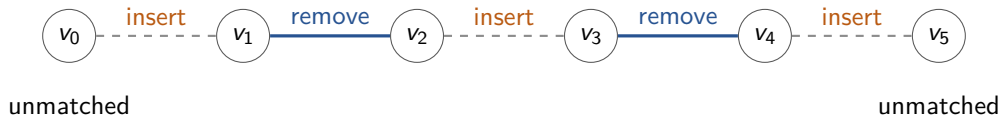
No edge of P touches x , so **no incident edge changes**.

A matched vertex stays matched.

An unmatched vertex stays unmatched.

Why the matching grows by one edge

An augmenting path starts and ends at unmatched vertices, so its **first and last edges lie outside M** .



Example: remove **2** edges and insert **3** edges.

In general, alternation gives **one more insertion than removal**:

$$|M'| = |M| - \underbrace{r}_{\text{removed}} + \underbrace{(r+1)}_{\text{inserted}} = |M| + 1.$$

Together: The flip preserves a matching and increases its size by one.

Notation: $M' = M \triangle E(P)$ toggles exactly the path edges.

Berge's theorem

Characterization

A matching M is maximum if and only if it has no M -augmenting path.

An M -augmenting path alternates between edges outside and inside M and has both endpoints unmatched by M .

Proof ($\Rightarrow$). Suppose P is an M -augmenting path. Replacing the M -edges of P with its the flip gives a matching of size $|M| + 1$. Hence M is not maximum which leads to a contradiction.

Berge's theorem

Proof ($\Leftarrow$), by contrapositive. Suppose M is not maximum, and let M^* be a maximum matching. Then $|M^*| > |M|$. Consider their symmetric difference

$$M \Delta M^* = (M \setminus M^*) \cup (M^* \setminus M),$$

the edges belonging to exactly one of the two matchings.

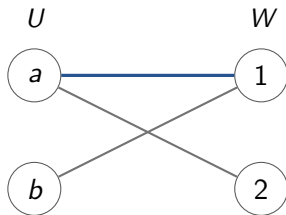
- 1 Since M, M^* are both matching, every vertex has degree at most two in $M \Delta M^*$. Its nontrivial components are paths or cycles whose edges alternate between M and M^* .
- 2 Each cycle has equally many edges from the two matchings as a bipartite graph don't have odd cycle. Since $|M^*| > |M|$, some path has more M^* -edges than M -edges.
- 3 By alternation, that path starts and ends with M^* -edges. Its endpoints are unmatched by M , so it augments M .

Thus a matching with no augmenting path must be maximum.

Example

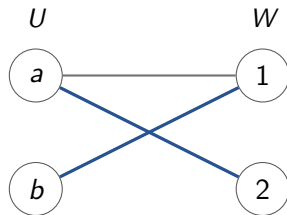
Suppose M is not maximum. Choose a maximum matching M^* , so $|M^*| > |M|$.

The current matching M



$$M = \{(a, 1)\}, \quad |M| = 1$$

A maximum matching M^*



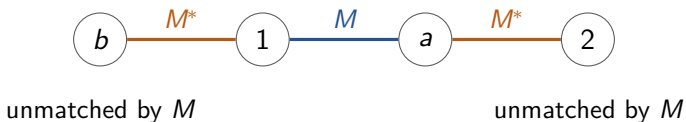
$$M^* = \{(a, 2), (b, 1)\}, \quad |M^*| = 2$$

In this graph, M^* has one more edge than M .

The edges that differ form an alternating path

The **symmetric difference** keeps edges in exactly one matching:

$$M \triangle M^* = (M \setminus M^*) \cup (M^* \setminus M) = \{(b, 1), (a, 1), (a, 2)\}.$$



The path $b, 1, a, 2$ has two M^* -edges and one M -edge. It is an M -augmenting path.

Finding an augmenting path by reachability

Give the bipartite graph a temporary orientation:

$$e \notin M : U \longrightarrow W,$$

$$e \in M : W \longrightarrow U.$$

Run BFS from **all unmatched vertices in U** . Stop if it reaches an unmatched vertex in W .

For $M = \{(a, 1)\}$, the directed path is

$$b \longrightarrow 1 \longrightarrow a \longrightarrow 2.$$

Direction forces the alternating pattern. BFS parent pointers recover the path to flip.

The matching decision algorithm

On input $\langle G, k \rangle$:

- ① If $k = 0$, accept. If $k > \min\{|U|, |W|\}$, reject.
- ② Set $M \leftarrow \emptyset$.
- ③ While $|M| < k$:
 - ① Orient the edges relative to the current M .
 - ② Search for an augmenting path using BFS.
 - ③ If none exists, reject and halt.
 - ④ Otherwise flip that path to increase $|M|$ by one.
- ④ Accept.

Recompute or update the orientation after every augmentation.

Correctness and polynomial running time

Acceptance: Each flip preserves a matching, so reaching size k certifies a yes-instance.

Rejection: If no augmenting path exists, Berge's theorem says the current matching is maximum. Its size is still below k .

There are at most $\min\{|U|, |W|\}$ augmentations. A BFS and flip cost $O(v + m)$ graph operations.

$$T = O(v(v + m)), \quad \text{BIPARTITE-MATCHING} \in \text{P}.$$

Algorithm chain: Reachability finds an augmenting path, and augmentation builds a sufficiently large matching.

Example: Split Array Largest Sum

Given nonnegative integers $A = (a_1, \dots, a_n)$ and $1 \leq k \leq n$, split A into exactly k nonempty **contiguous** blocks. Minimize the largest block sum.

$$\underbrace{[7, 2, 5]}_{14} \mid \underbrace{[10, 8]}_{18} \quad (k = 2).$$

- **LeetCode's output:** the smallest possible largest sum, here 18.
- **Our search output:** the actual cut positions, here a cut after a_3 .

First, fix a proposed cap B and find a partition that meets it. Then we will find the smallest feasible cap.

Problem: [LeetCode 410, Split Array Largest Sum](#).

Decision and search use the same input

Fix $x = \langle A, k, B \rangle$.

Decision problem $D(A, k, B)$

Does a partition into exactly k nonempty contiguous blocks exist, with every block sum at most B ? Return **yes** or **no**.

Search problem $S(A, k, B)$

Output the $k - 1$ cut positions of such a partition, or output $\perp$ if no such partition exists.

The cuts form a **witness**: check their order, nonempty blocks, and block sums.

$$D(A, k, B) = 1 \iff \text{a valid witness for } S(A, k, B) \text{ exists.}$$

Goal: Solve S using calls to D , even if D reveals no cuts.

A linear-time decision procedure

Greedy rule: Extend the current block while its sum stays at most B .

Algorithmic description

On input $\langle A, k, B \rangle$, where $A = (a_1, \dots, a_n)$, $a_i \geq 0$, and $1 \leq k \leq n$:

- ① Initialize the block count $g \leftarrow 1$ and the current block sum $s \leftarrow 0$.
- ② For each entry a_i , from left to right:
 - ① If $a_i > B$, reject and halt.
 - ② If $s + a_i > B$, start a new block: set $g \leftarrow g + 1$ and $s \leftarrow a_i$.
 - ③ Otherwise, extend the current block: set $s \leftarrow s + a_i$.
- ③ Accept if $g \leq k$; otherwise, reject.

The scan uses $O(n)$ arithmetic operations and $O(1)$ auxiliary words.

Why the greedy decision procedure works

Fix $B \geq \max_i a_i$. Let g_j be the endpoint of greedy's j th block, and p_j the endpoint of any feasible partition's j th block.

Greedy stays ahead

After j blocks, greedy has covered at least as many entries: $g_j \geq p_j$, unless greedy has already finished the array.

- Initially $g_0 = p_0 = 0$.
- If $g_{j-1} \geq p_j$, the claim already holds.
- Otherwise $p_{j-1} \leq g_{j-1} < p_j$. Nonnegativity gives $\sum_{i=g_{j-1}+1}^{p_j} a_i \leq \sum_{i=p_{j-1}+1}^{p_j} a_i \leq B$. Greedy can extend at least to p_j .

Thus greedy uses no more blocks than any feasible partition.

The decision answers at $B = 17$ and $B = 18$

Array: $[7, 2, 5, 10, 8]$, $k = 2$.

Cap	Greedy blocks	Count	Answer
17	$[7, 2, 5] \mid [10] \mid [8]$	3	no
18	$[7, 2, 5] \mid [10, 8]$	2	yes

At 17, the last two entries cannot share a block. Greedy's minimality proves that no two-block partition meets the cap.

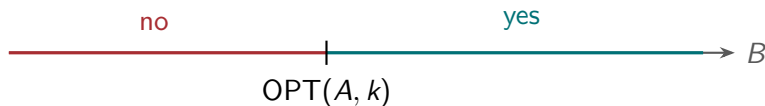
At 18, the displayed partition witnesses feasibility. Hence the integer optimum is 18.

The threshold decision problem

Let $D(A, k, B)$ answer:

Can A be split into exactly k nonempty contiguous blocks, each with sum at most B ?

If a partition works for B , the same partition works for every $B' \geq B$.



$$\text{OPT}(A, k) = \min\{B : D(A, k, B) = 1\}.$$

The monotone object is the **answer to the threshold question**. The array itself need not be sorted.

A competitive-programming trick in disguise

“Binary search on the answer”

Instead of directly finding the best value, repeatedly ask: Is there a solution whose cost is at most B ?

For a feasible integer minimization instance x , define

$$F_x(B) = 1 \iff \exists w \text{ feasible for } x \text{ with } \text{cost}(w) \leq B.$$

The answers switch from no to yes at the optimum.

This is an **optimization-to-decision reduction**. It fits the search-to-decision viewpoint: use decision queries to find the answer, then recover a witnessing solution when one is required.

Binary search for the first feasible cap

Let $L = \max_i a_i$ and $U = \sum_i a_i$. Nonnegativity gives $L \leq \text{OPT}(A, k) \leq U$.

Algorithmic description

On input $\langle A, k \rangle$:

- ① Initialize $\ell \leftarrow L$ and $h \leftarrow U$.
- ② While $\ell < h$:
 - ① Set $B \leftarrow \ell + \lfloor (h - \ell)/2 \rfloor$.
 - ② Query $D(A, k, B)$ using the greedy decision procedure.
 - ③ If the answer is yes, set $h \leftarrow B$. Otherwise, set $\ell \leftarrow B + 1$.
- ③ Output ℓ .

Invariant: $\text{OPT}(A, k) \in [\ell, h]$. Each iteration strictly shrinks the interval. At termination, $\ell = h = \text{OPT}(A, k)$.

Binary-search trace

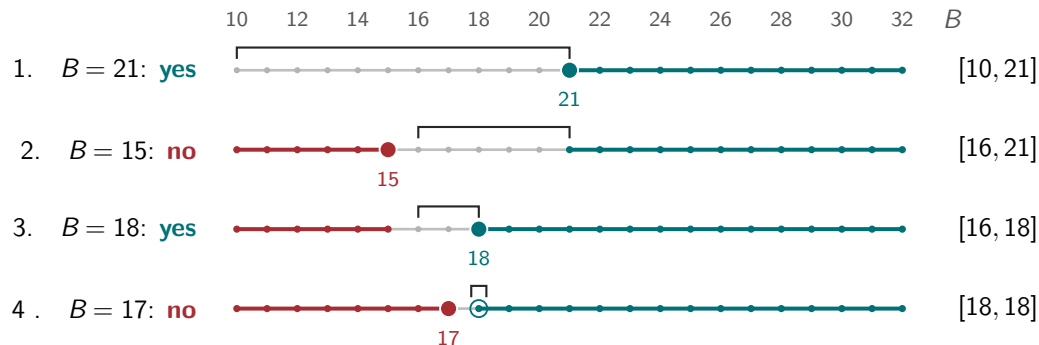
For $A = [7, 2, 5, 10, 8]$ and $k = 2$, initially $[L, U] = [10, 32]$.

Current interval	Midpoint	Decision	Next interval
$[10, 32]$	21	yes	$[10, 21]$
$[10, 21]$	15	no	$[16, 21]$
$[16, 21]$	18	yes	$[16, 18]$
$[16, 18]$	17	no	$[18, 18]$

$$\text{OPT}(A, 2) = 18$$

Four one-bit answers locate the optimum in a range of 23 candidate values.

Graphical Illustration



$$\text{OPT}(A, 2) = 18$$

Four queries leave one candidate.

Recovering an optimal partition

Once binary search returns B^* :

- 1 Rerun the greedy scan with cap B^* , recording its cut positions.
- 2 If it uses fewer than k blocks, split nonsingleton blocks until exactly k remain.

Every block still has sum at most B^* . If the largest sum were smaller than B^* , binary search would not have returned the optimum.

$[7, 2, 5] \mid [10, 8]$ is an optimal witness.

Value versus witness: Binary search identifies B^* . Recording and refining the blocks supplies the actual solution.

Recovering the cuts using only decision queries (Search-to-Decision Reduction)

Suppose D reveals only a bit. After finding B^* , let the remaining suffix start at i and require r blocks.

- 1 Try endpoints $j = i, \dots, n - r + 1$ for the next block.
- 2 Keep only candidates with $\sum_{t=i}^j a_t \leq B^*$.
- 3 Ask $D(A[j+1..n], r-1, B^*)$. On yes, record the block $A[i..j]$, set $i \leftarrow j+1$, and decrease r by one.

Define $D(\emptyset, 0, B) = 1$ and all other zero-block or empty-array requests as no.

The same extension question as for matching

Can this first block be completed to a feasible partition of the suffix? The invariant is that the remaining suffix admits exactly r blocks.

Finding the first cut with yes/no questions

Let $A = [7, 2, 5, 10, 8, 10, 8]$ and $k = 3$. Suppose binary search has returned $B^* = 18$.

Start with $i = 1$ and $r = 3$. Try a first block and ask whether the **remaining suffix can form two blocks** under cap 18.

First block	Sum	Query to D	Answer
$[7]$	7	$D([2, 5, 10, 8, 10, 8], 2, 18)$	no
$[7, 2]$	9	$D([5, 10, 8, 10, 8], 2, 18)$	no
$[7, 2, 5]$	14	$D([10, 8, 10, 8], 2, 18)$	yes

Record the first cut after position 3:

$$\underbrace{[7, 2, 5]}_{\text{fixed first block}} \mid \underbrace{[10, 8, 10, 8]}_{\text{two blocks still needed}} .$$

Update $i \leftarrow 4$ and $r \leftarrow 2$. The yes-answer guarantees a feasible completion of this suffix.

Finding the remaining blocks with yes/no questions

The first block is fixed as $[7, 2, 5]$. Continue on the suffix $[10, 8, 10, 8]$, still with cap 18.

State	Next block	Sum	Query to D	Answer
$i = 4, r = 2$	$[10]$	10	$D([8, 10, 8], 1, 18)$	no
	$[10, 8]$	18	$D([10, 8], 1, 18)$	yes
$i = 6, r = 1$	$[10]$	10	$D([8], 0, 18)$	no
	$[10, 8]$	18	$D(\emptyset, 0, 18)$	yes

Updates after yes: $(i, r) = (4, 2) \rightarrow (6, 1) \rightarrow (8, 0)$. The final suffix is empty.

$$\underbrace{[7, 2, 5]}_{14} \mid \underbrace{[10, 8]}_{18} \mid \underbrace{[10, 8]}_{18}.$$

Output: cuts after positions 3 and 5.

Why the number of queries is polynomial

There are $U - L + 1$ candidate integer caps. Binary search makes at most

$$\lceil \log_2(U - L + 1) \rceil$$

queries. Including initialization, the greedy implementation costs

$$O(n(1 + \log(U - L + 1)))$$

arithmetic operations.

If each entry uses at most b bits, then $U < n2^b$ and caps and sums use $O(b + \log n)$ bits. Thus the query count and bit costs are polynomial in the encoded input length.

Optional black-box witness recovery adds at most kn decision queries.

Takeaways

- ① Design polynomial-time algorithms and rigorously analyze their running time in terms of the input length.
- ② Decision algorithms are often more powerful than you might expect.
- ③ Ideas from CSCI3360 can help you solve LeetCode problems!